main_android.dart 쉬운 해설

대상 파일: project_canvas/lib/main_android.dart

목표: Flutter를 처음 보는 사람도 파일을 위에서 아래로 따라가며 "이 줄이 왜 있는지" 이해할 수 있게 설명

이 파일의 정체 이 파일은 단순한 화면 파일이 아닙니다. 앱 실행, 메인 화면, 캔버스 그리기, 배경 이미지 업로드, 좌표 변환, Z 값 관리, TCP 전송, 웹뷰까지 거의 전부 들어 있습니다.

한 줄 요약 이미지 위에 경로를 그리고, 그 경로를 점 좌표로 바꾼 다음, mm 단위와 Z 값을 붙여 JSON으로 서버에 보내는 도구입니다.

1. import: 어떤 도구를 가져오는가

```
import 'dart:convert';
import 'dart:io';
import 'dart:math' as math;

import 'package:flutter/gestures.dart';
import 'package:image_picker/image_picker.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:webview_flutter/webview_flutter.dart';
import 'package:flutter/foundation.dart';
import 'package:url_launcher/url_launcher.dart';
import 'package:webview_windows/webview_windows.dart';
```

dart:convert: JSON 인코딩에 사용합니다. 나중에 좌표 데이터를 jsonEncode로 보내기 위해 필요합니다.

dart:io: 파일, 소켓, 네트워크 인터페이스, 플랫폼 구분 같은 OS 기능을 쓰기 위해 필요합니다.

dart:math as math: 원 둘레 계산, 최소/최대 계산, 삼각함수 등에 씁니다.

flutter/gestures: 마우스 버튼 종류 같은 pointer 입력 처리에 씁니다.

image_picker: 갤러리에서 이미지를 고를 때 씁니다.

material.dart: Flutter UI의 기본 재료입니다.

services.dart: 키보드 상태, 예를 들면 Shift 키가 눌렸는지 확인할 때 씁니다.

webview_flutter, webview_windows: 웹 페이지를 앱 안에서 보여줄 때 씁니다.

foundation.dart: 현재 플랫폼이 Android, iOS, Windows인지 구분할 때 씁니다.

url_launcher: 브라우저를 외부로 여는 데 씁니다.

2. 앱 시작

```
void main() {
runApp(const CanvasApp());
}
```

main(): 프로그램 시작점입니다.

runApp(...): Flutter에게 "이 위젯을 루트로 앱을 시작해"라고 말합니다.

const CanvasApp(): 최상위 앱 위젯을 생성합니다.

3. CanvasApp: 앱의 껍데기

```
class CanvasApp extends StatelessWidget {
const CanvasApp({super.key});

@override
Widget build(BuildContext context) {
return MaterialApp(
title: 'Canvas Line Sender',
theme: ThemeData(
colorScheme: ColorScheme.fromSeed(seedColor: Colors.blueGrey),
useMaterial3: true,
),
home: const CanvasHomePage(),
);
}
}
```

extends StatelessWidget: 이 위젯 자체는 변하는 상태를 들고 있지 않는다는 뜻입니다.

const CanvasApp({super.key}): Flutter가 필요할 때 이 위젯을 식별할 수 있도록 key를 부모에 전달합니다.

build(...): 실제로 어떤 UI를 보여줄지 반환하는 함수입니다.

MaterialApp: 앱 전체 공통 설정을 담당하는 루트 위젯입니다.

title: 앱 제목입니다. 브라우저 탭 제목이나 앱 전환 화면에서 쓰일 수 있습니다.

theme: 앱 전체 색과 기본 머티리얼 스타일을 결정합니다.

home: 앱을 열었을 때 가장 먼저 보여줄 화면입니다. 여기서는 CanvasHomePage입니다.

4. CanvasHomePage와 createState

```
class CanvasHomePage extends StatefulWidget {
const CanvasHomePage({super.key});

@override
State<CanvasHomePage> createState() => _CanvasHomePageState();
}
```

StatefulWidget: 화면이 계속 바뀌는 위젯입니다. 이 앱은 그리고, 선택하고, 보내고, 상태 문구가 계속 바뀌므로 StatefulWidget이 맞습니다.

createState(): 이 화면의 실제 상태를 관리할 객체를 만듭니다.

=>: 짧은 return 문법입니다. { return _CanvasHomePageState(); }와 같은 뜻입니다.

5. _CanvasHomePageState: 진짜 핵심 본체

이 클래스 안에 변수, 이벤트 처리, UI, 전송 로직이 대부분 들어 있습니다. 아래 변수들은 성격별로 묶어서 읽는 것이 이해하기 쉽습니다.

5-1. 기본 설정값

```
static const double _defaultStepPx = 10.0;
static const double _defaultMmPerPx = 1.0;
static const String _defaultHost = '127.0.0.1';
static const int _defaultPort = 9000;
```

static const: 클래스 차원의 고정값입니다. 실행 중 바뀌지 않는 기본 설정입니다.

_defaultStepPx: 점 샘플링 간격 기본값입니다.

_defaultMmPerPx: 1픽셀이 몇 mm인지 기본 비율입니다.

_defaultHost, _defaultPort: TCP 연결 기본값입니다.

5-2. 지금 그리고 있는 도형 상태

```
Offset? _p0;
Offset? _p1;
List<Offset> _rawPoints = <Offset>[];
bool _isDrawing = false;
```

Offset: x, y 좌표 한 점입니다.

_p0: 시작점입니다.

_p1: 현재 끝점 또는 마지막 점입니다.

_rawPoints: pen 모드에서 손이 지나간 원본 좌표 목록입니다.

_isDrawing: 현재 드래그로 그리고 있는 중인지 표시합니다.

5-3. 샘플링과 좌표 변환 값

```
double _stepPx = _defaultStepPx;
double _mmPerPx = _defaultMmPerPx;

List<Offset> _pointsPx = <Offset>[];
List<Offset> _pointsMm = <Offset>[];
List<Offset> _deltasMm = <Offset>[];
List<Offset> _pointsImage = <Offset>[];
```

_stepPx: 점과 점 사이 간격입니다. 작을수록 포인트 수가 많아집니다.

_mmPerPx: 이미지 좌표를 실제 길이(mm)로 바꾸는 비율입니다.

_pointsPx: 화면 캔버스 좌표 기준 샘플 점들입니다.

_pointsImage: 원본 이미지 좌표 기준 샘플 점들입니다.

_pointsMm: 최종 전송에 가까운 mm 단위 좌표들입니다.

_deltasMm: 점과 다음 점 사이 이동량입니다.

5-4. Z 값

```
List<double> _currentZValues = <double>[];
```

각 점마다 Z 값을 저장합니다. X, Y만 보내는 것이 아니라 X, Y, Z 세 값을 보내기 때문입니다.

5-5. stroke 관리

```
final List<Stroke> _strokes = <Stroke>[];
int? _dragStrokeIndex;
Offset? _dragLast;
bool _isMoving = false;
int? _selectedStrokeIndex;
bool _appendPen = false;
int? _appendStrokeIndex;
```

Stroke: 한 번 그린 선 묶음입니다.

_strokes: 지금까지 확정된 모든 stroke를 담습니다.

_dragStrokeIndex: 현재 이동 중인 stroke 번호입니다.

_dragLast: 이전 마우스 위치입니다. 이동량 계산에 씁니다.

_isMoving: stroke를 움직이는 중인지 표시합니다.

_selectedStrokeIndex: 현재 선택된 stroke 번호입니다.

_appendPen: pen 모드에서 이전 stroke 뒤에 이어서 붙일지 정합니다.

_appendStrokeIndex: 실제로 이어붙이고 있는 대상 stroke 번호입니다.

5-6. 선택, 편집, 스무딩

```
bool _selectMode = false;
bool _isSelecting = false;
Offset? _selectStart;
Rect? _selectRect;
```

```
int _smoothingWindow = 5;
int? _smoothingBaseStrokeIndex;
```

```
bool _isEditingPoint = false;
int? _editStrokeIndex;
int? _editSampleIndex;
int? _editPathIndex;
```

_selectMode: 박스 선택 모드 on/off입니다.

_isSelecting: 현재 드래그로 선택 박스를 그리고 있는지 표시합니다.

_selectStart, _selectRect: 선택 박스 시작점과 현재 사각형입니다.

_smoothingWindow: 얼마나 넓게 평균낼지 정합니다.

_isEditingPoint: Shift를 누르고 점 하나를 직접 편집 중인지 표시합니다.

_editStrokeIndex, _editSampleIndex, _editPathIndex: 어느 stroke의 어느 점을 편집 중인지 기억합니다.

5-7. 현재 그리기 모드

```
DrawMode _mode = DrawMode.line;
```

앱 시작 시에는 선 모드로 시작합니다. line, pen, circle, rect 중 하나가 들어갑니다.

5-8. 배경 이미지와 프레임

```
File? _imageFile;
Size? _imageSize;
Size? _userFrameMmSize;
Size? _userFramePxSize;
double _userFrameScale = 2.3;
bool _useUserFrameOverride = true;
Rect? _frameRect;
final ImagePicker _imagePicker = ImagePicker();
```

_imageFile: 업로드한 배경 이미지 파일입니다.

_imageSize: 이미지 원본 너비와 높이입니다.

_userFrameMmSize: 사용자가 입력한 실제 프레임 크기(mm)입니다.

_userFramePxSize: 프레임 표시 계산에 쓰는 논리 크기입니다.

_userFrameScale: 프레임을 화면에 얼마나 크게 보여줄지 정합니다.

_useUserFrameOverride: 이미지 크기 대신 사용자가 넣은 프레임 크기를 우선할지 정합니다.

_frameRect: 화면 안에서 실제 프레임이 차지하는 사각형 위치입니다.

_imagePicker: 갤러리에서 이미지를 선택하는 도구입니다.

5-9. 입력 컨트롤러

```
final TextEditingController _hostController =
TextEditingController(text: _defaultHost);
final TextEditingController _portController =
TextEditingController(text: _defaultPort.toString());
final TextEditingController _zController = TextEditingController(text: '0.0');
final TextEditingController _stepPxController =
TextEditingController(text: _defaultStepPx.toStringAsFixed(1));
final TextEditingController _mmPerPxController =
TextEditingController(text: _defaultMmPerPx.toStringAsFixed(3));
final TextEditingController _bgWidthController = TextEditingController();
final TextEditingController _bgHeightController = TextEditingController();
```

TextField와 코드 사이를 연결하는 객체입니다. 사용자가 입력창에 넣은 글자를 읽거나, 코드가 입력창 초기값을 넣을 때 사용합니다.

_hostController, _portController: TCP 연결 주소 입력용입니다.

_zController: Z 값 입력용입니다.

_stepPxController: 점 간격 입력용입니다.

_mmPerPxController: mm/px 비율 입력용입니다.

_bgWidthController, _bgHeightController: 배경 실제 크기 입력용입니다.

5-10. 소켓 상태

```
Socket? _socket;
String? _connectedHost;
int? _connectedPort;
String _status = 'Ready';
```

_socket: 현재 열려 있는 TCP 연결입니다.

_connectedHost, _connectedPort: 어디에 연결되었는지 기억합니다.

_status: 화면에 보여줄 상태 문구입니다.

6. 네트워크 보조 함수

```
Future<String> _networkDebugInfo() async { ... }
List<int>? _tryParseIpv4(String value) { ... }
bool _same24Subnet(String a, String b) { ... }
Future<InternetAddress?> _selectSourceAddress(String host) async { ... }
```

_networkDebugInfo(): 현재 장치의 네트워크 인터페이스 이름과 IP를 문자열로 모읍니다. 연결 실패 시 진단 정보로 씁니다.

_tryParseIpv4(): 192.168.0.10 같은 문자열을 [192, 168, 0, 10] 배열로 바꿉니다.

_same24Subnet(): 두 IP의 앞 세 구간이 같은지 비교해 같은 대역인지 확인합니다.

_selectSourceAddress(): 장치에 IP가 여러 개 있을 때, 목표 host와 같은 대역의 로컬 IP를 우선 선택합니다.

이 블록의 목적은 "그냥 연결"만 하는 것이 아니라, 연결 실패 시 원인을 더 잘 보이고, 여러 네트워크 인터페이스가 있을 때 조금 더 안정적으로 연결하려는 것입니다.

7. initState와 dispose

```
@override
void initState() {
super.initState();
_bgWidthController.text = '594.0';
_bgHeightController.text = '420.0';
_userFrameMmSize = const Size(594.0, 420.0);
_userFramePxSize = const Size(594.0, 420.0);
_mmPerPx = 1.0;
_mmPerPxController.text = _mmPerPx.toStringAsFixed(3);
}
```

initState(): 이 화면이 처음 생길 때 한 번 실행됩니다.

기본 프레임을 A2 크기인 594 x 420으로 잡아 입력창과 내부 상태를 같이 초기화합니다.

_mmPerPx = 1.0: 처음에는 1픽셀을 1mm처럼 다루는 단순 기준으로 시작합니다.

```
@override
void dispose() {
_hostController.dispose();
_portController.dispose();
_zController.dispose();
_stepPxController.dispose();
_mmPerPxController.dispose();
_bgWidthController.dispose();
_bgHeightController.dispose();
_socket?.destroy();
super.dispose();
}
```

dispose(): 이 화면이 사라질 때 마지막으로 정리하는 함수입니다.

TextEditingController는 내부 상태와 리스너를 들고 있으므로 화면이 끝날 때 해제해야 합니다.

_socket?.destroy(): 열려 있는 TCP 연결이 있다면 종료합니다.

super.dispose(): 부모 State의 정리 작업도 실행합니다.

8. 사용자가 입력한 설정 적용 함수

```
void _applyMmPerPx() { ... }
void _applyStepPx() { ... }
void _applyZToAll() { ... }
void _applyBackgroundSize() { ... }
void _setPresetSize(double widthMm, double heightMm) { ... }
```

_applyMmPerPx(): 사용자가 입력한 mm/px 값을 읽고 내부 상태에 적용합니다. 잘못된 값이면 status에 오류를 표시합니다.

_applyStepPx(): 점 간격을 바꾸고, 현재 그리고 있던 도형이 있으면 즉시 다시 계산합니다.

_applyZToAll(): 현재 선택된 stroke 또는 전체 stroke의 모든 점에 같은 Z 값을 넣습니다.

_applyBackgroundSize(): 사용자가 입력한 배경 실제 크기(mm)를 프레임 기준으로 적용합니다.

_setPresetSize(): A4, A2, B4 같은 프리셋 버튼을 눌렀을 때 호출됩니다.

9. 포인터 입력: 그리기, 이동, 선택, 점 편집

```
void _onPointerDown(PointerDownEvent event) { ... }
void _onPointerMove(PointerMoveEvent event) { ... }
void _onPointerUp(PointerUpEvent event) { ... }
```

_onPointerDown

- 우클릭이면 stroke 이동 시작
- Shift + pen 모드면 점 편집 시작
- Select 모드면 선택 박스 시작
- 그 외에는 새 도형 그리기 시작

_onPointerMove

- 이동 중이면 stroke 전체 이동
- 점 편집 중이면 선택한 점 위치 수정
- 선택 중이면 사각형 갱신
- 그리기 중이면 현재 도형 갱신 후 점 다시 계산

_onPointerUp

- 이동 중이었으면 이동 종료
- 점 편집 중이었으면 편집 종료
- 선택 중이었으면 선택 박스로 stroke 선택
- 그리기 중이었으면 점을 확정하고 stroke 목록에 저장

이 세 함수가 사용자 입력의 중심입니다. 이 앱은 버튼보다 캔버스 상호작용이 중요하므로 이 블록이 매우 핵심입니다.

10. 현재 상태 지우기와 삭제

```
void _reset() { ... }
void _clearCurrentDrawing() { ... }
void _deleteSelectedOrLast() { ... }
```

_reset(): 전체를 초기화합니다. 그린 것, 선택 상태, 임시 점, stroke 목록을 거의 모두 비웁니다.

_clearCurrentDrawing(): 지금 그리고 있던 임시 도형만 지웁니다.

_deleteSelectedOrLast(): 선택된 stroke가 있으면 그 stroke를 지우고, 없으면 마지막 stroke를 지웁니다.

11. 이미지 업로드

```
Future<void> _pickImage() async {
final picked = await _imagePicker.pickImage(source: ImageSource.gallery);
if (picked == null) return;
final bytes = await picked.readAsBytes();
final decoded = await decodeImageFromList(bytes);
setState(() {
_imageFile = File(picked.path);
_imageSize = Size(decoded.width.toDouble(), decoded.height.toDouble());
...
});
}
```

갤러리에서 이미지를 선택합니다.

이미지를 메모리로 읽은 뒤 실제 너비와 높이를 구합니다.

_imageFile과 _imageSize를 저장하면 이후 캔버스에 이미지를 배경으로 깔 수 있습니다.

12. TCP 연결 만들기

```
Future<void> _ensureConnected() async { ... }
```

Host와 Port 입력값이 유효한지 먼저 검사합니다.

이미 같은 host, port에 연결된 소켓이 있으면 재사용합니다.

다른 곳에 연결 중이면 기존 소켓을 닫고 새 연결을 만듭니다.

Socket.connect(...)로 실제 TCP 연결을 시도합니다.

socket.listen(...)으로 서버가 보내는 응답도 받아 status에 반영합니다.

에러가 나면 네트워크 인터페이스 정보까지 함께 상태 메시지에 적어 디버깅을 돕습니다.

13. 전송: JSON 만들고 보내기

```
Future<void> _sendJson() async { ... }
Map<String, dynamic> _buildPayload() { ... }
int _payloadPointCount() { ... }
```

_sendJson(): 현재 그린 도형 또는 stroke가 있는지 검사한 뒤, 연결을 확보하고 JSON을 전송합니다.

_buildPayload(): 실제 전송할 데이터 구조를 만듭니다.

_payloadPointCount(): 몇 개의 포인트가 전송되는지 UI 표시용으로 계산합니다.

전송 형식

```
{
"points": [
[12.34, 56.78, 0.00],
[14.11, 58.40, 0.00],
["000"]
]
}
```

[x, y, z] 하나가 점 하나입니다. stroke가 여러 개일 때는 stroke 끝마다 ["000"] 마커를 추가해 구분합니다.

14. 가장 중요한 함수: _rebuildPoints

```
void _rebuildPoints() { ... }
```

이 함수는 "사용자가 그린 모양"을 "전송 가능한 점들"로 바꿉니다.

모드에 따라 다르게 처리합니다.

- line: 시작점과 끝점 사이를 일정 간격으로 나눔
- circle: 원 둘레를 일정 간격으로 샘플링
- rect: 사각형 테두리를 따라 샘플링
- pen: 사용자가 그린 자유곡선을 다시 일정 간격으로 재샘플링

그 뒤 _pointsImage, _pointsPx, _pointsMm, _deltasMm를 모두 갱신합니다.

즉 이 함수가 있어야 "보이는 선"이 "실제 데이터"로 바뀝니다.

15. 좌표 계산과 샘플링 수학 함수

```
List<Offset> _applyTransform(List<Offset> points, double scale) { ... }
List<Offset> _buildDeltas(List<Offset> points) { ... }
List<Offset> _resamplePolyline(List<Offset> input, double stepPx) { ... }
List<Offset> _circlePoints(Offset center, Offset edge, double stepPx) { ... }
List<Offset> _rectPoints(Offset a, Offset b, double stepPx) { ... }
List<Offset> _sampleSegment(Offset a, Offset b, double stepPx) { ... }
```

_applyTransform(): 이미지 좌표에 scale을 적용해 mm 좌표로 바꿉니다. 필요하면 y 뒤집기나 회전도 넣을 수 있게 구조가 잡혀 있습니다.

_buildDeltas(): 각 점과 다음 점 사이의 차이 벡터를 계산합니다.

_resamplePolyline(): 자유곡선을 일정 간격으로 다시 찍습니다.

_circlePoints(): 원 둘레 샘플 점 생성 함수입니다.

_rectPoints(): 사각형 테두리 샘플 점 생성 함수입니다.

_sampleSegment(): 한 직선 구간을 일정 간격으로 쪼갭니다.

16. build(): 화면 배치의 중심

```
@override
Widget build(BuildContext context) {
final lengthPx = _lengthForMode();
final mediaQuery = MediaQuery.of(context);
final isLandscape = mediaQuery.orientation == Orientation.landscape;
...
return Scaffold(
appBar: AppBar(title: const Text('Canvas Line Sender')),
body: LayoutBuilder(
builder: (context, constraints) {
final canvas = _buildCanvasArea();
final controls = _buildControlsPanel(...);
if (isLandscape) {
return Row(...);
}
return Column(...);
},
),
);
}
```

Scaffold: 머티리얼 앱 기본 화면 구조입니다. AppBar와 Body를 가집니다.

_buildCanvasArea(): 그림 그리는 영역을 만듭니다.

_buildControlsPanel(): 버튼과 입력창이 모여 있는 제어 패널을 만듭니다.

가로 화면이면 좌우 분할, 세로 화면이면 위아래 분할로 배치합니다.

17. 캔버스 영역 만들기

Widget _buildCanvasArea() { ... }

LayoutBuilder로 사용 가능한 화면 크기를 받습니다.

그 크기 안에서 프레임 크기를 계산합니다.

_frameRect를 설정해 "이미지가 실제로 어디에 그려지는지" 저장합니다.

아래 레이어에는 이미지, 위 레이어에는 선과 점을 그리는 CustomPaint가 올라갑니다.

Listener가 pointer down, move, up을 받아 캔버스 상호작용을 처리합니다.

18. 제어 패널 만들기

Widget _buildControlsPanel(...) { ... }

오른쪽 또는 아래에 붙는 조작판입니다.

- 현재 시작점, 끝점, 길이, 포인트 수, 상태 표시
- Points Table 표시 및 Z 수정
- Host, Port 입력
- Reset, Test TCP, Send, Delete, Upload, Onshape 버튼
- Line, Pen, Circle, Rect 모드 전환
- Continue, Select 토글
- Point gap, Smoothing, BG Size, Frame Scale, mm/px 설정

즉 사용자가 앱 기능 대부분을 조작하는 메인 패널입니다.

19. 표에 보여줄 데이터 만들기

```
List<DataRow> _buildPointRows() { ... }
List<Offset> _pointsForTable() { ... }
List<double> _zValuesForTable(int length) { ... }
List<_TablePointRow> _tableRows() { ... }
```

현재 선택된 stroke가 있으면 그 stroke만 표에 보여줍니다.

선택된 stroke가 없고 stroke가 여러 개면 전체 stroke를 순회하며 행을 만듭니다.

_TablePointRow는 표의 한 줄을 표현하는 작은 데이터 모델입니다.

표에서 Z 칸은 TextFormField라서 사용자가 직접 값을 고칠 수 있습니다.

20. 길이 계산

```
double _polylineLength(List<Offset> points) { ... }
double _lengthForMode() { ... }
```

_polylineLength(): 점 목록의 총 길이를 구합니다.

_lengthForMode(): 현재 모드가 line, circle, rect, pen 중 무엇인지에 따라 적절한 길이를 계산합니다.

21. 좌표계 변환: 화면 좌표와 이미지 좌표를 오가기

```
Offset _canvasToImageClamped(Offset p) { ... }
Offset _imageToCanvas(Offset p) { ... }
List<Offset> _imagePointsToCanvas(List<Offset> points) { ... }
```

_canvasToImageClamped(): 사용자가 화면에서 찍은 점을 이미지 내부 좌표로 변환합니다.

_imageToCanvas(): 이미지 좌표를 다시 화면 좌표로 바꿉니다.

_imagePointsToCanvas(): 점 목록 전체를 한 번에 변환합니다.

이 앱에서 이 부분이 중요한 이유는, 사용자가 보는 화면 크기와 원본 이미지 크기가 다를 수 있기 때문입니다. 화면에서 100px 이동했다고 원본 이미지에서도 100px 이동은 아닐 수 있습니다.

22. stroke 이동과 점 편집

```
void _beginMove(Offset position) { ... }
void _updateMove(Offset position) { ... }
void _endMove() { ... }
```

```
bool _isShiftPressed() { ... }
bool _beginEditPoint(Offset position) { ... }
void _updateEditPoint(Offset position) { ... }
void _endEditPoint() { ... }
```

우클릭하면 가장 가까운 stroke를 찾고 이동을 시작합니다.

드래그 중에는 이전 좌표와 현재 좌표 차이만큼 stroke 전체를 옮깁니다.

Shift가 눌린 상태에서는 개별 점 편집 모드로 들어갑니다.

점 편집은 sample과 path 둘 다 적절히 갱신해 화면 표시와 실제 데이터가 어긋나지 않게 합니다.

23. 히트 테스트: 무엇을 클릭했는지 찾기

```
_PointHit? _hitTestPoint(Offset position) { ... }
int? _nearestPathIndex(List<Offset> path, Offset position) { ... }
int? _hitTestStroke(Offset position) { ... }
double _minDistanceToPath(List<Offset> path, Offset p) { ... }
double _distanceToSegment(Offset p, Offset a, Offset b) { ... }
```

클릭한 좌표와 점 또는 선의 거리를 계산해 가장 가까운 대상을 찾습니다.

_distanceToSegment()는 한 점과 선분 사이 최소 거리를 구하는 기하 함수입니다.

이 함수들이 있어야 "사용자가 이 선을 집었다" 또는 "이 점을 잡았다"를 판별할 수 있습니다.

24. 선택 박스와 겹침 판정

```
void _selectStrokeByRect(Rect rect) { ... }
bool _strokeIntersectsRect(Stroke stroke, Rect rect) { ... }
bool _segmentIntersectsRect(Offset a, Offset b, Rect rect) { ... }
bool _segmentsIntersect(Offset p1, Offset p2, Offset q1, Offset q2) { ... }
```

드래그로 만든 선택 사각형과 stroke가 겹치는지 검사합니다.

단순히 점이 사각형 안에 있는지만 보는 것이 아니라 선분과 사각형 경계가 교차하는지도 확인합니다.

겹치는 stroke들 중 선택 박스 중심에 가장 가까운 stroke를 선택합니다.

25. smoothing

```
void _applySmoothingToSelected() { ... }
List<Offset> _smoothPoints(List<Offset> points) { ... }
```

선택된 stroke에 이동 평균 방식 스무딩을 적용합니다.

원본을 basePath, baseSamples에 보관해 슬라이더 값을 바꿔도 원본 기준으로 다시 계산할 수 있게 해둡니다.

_smoothPoints()는 주변 점들의 평균을 내서 경로를 더 부드럽게 만듭니다.

26. LinePainter: 실제로 화면에 그리기

```
class LinePainter extends CustomPainter { ... }
```

이 클래스는 상태를 직접 바꾸지 않고, 전달받은 데이터를 화면에 그리는 역할만 합니다.

stroke 선, sample 점, 점 번호, stroke 번호, 현재 그리고 있는 임시 도형, 선택 박스를 모두 그립니다.

선택된 stroke는 주황색으로 강조해 보여줍니다.

shouldRepaint는 이전 값과 지금 값이 다를 때 다시 그려야 하는지 알려줍니다.

27. enum과 작은 보조 클래스

```
enum DrawMode { line, pen, circle, rect }
```

DrawMode: 현재 도형 입력 방식 종류입니다.

```
class Stroke {
Stroke({required this.path, required this.samples, required this.zValues});
List<Offset> path;
List<Offset> samples;
List<double> zValues;
List<Offset>? basePath;
List<Offset>? baseSamples;
}
```

path: 실제 사용자가 그린 경로

samples: 일정 간격으로 재샘플링된 전송용 점들

zValues: sample 점마다 대응하는 Z 값

basePath, baseSamples: smoothing 원본 보관용

```
class _PointHit { ... }
class _TablePointRow { ... }
```

_PointHit: 히트 테스트 결과를 담는 작은 구조체입니다.

_TablePointRow: 좌표 표 한 행을 표현하는 데이터입니다.

28. WebViewerPage

```
class WebViewerPage extends StatefulWidget { ... }
class _WebViewerPageState extends State<WebViewerPage> { ... }
```

Onshape나 3D viewer 같은 사이트를 앱 안에서 여는 간단한 웹뷰 페이지입니다.

Windows에서는 webview_windows, 모바일에서는 webview_flutter를 사용합니다.

웹뷰 초기화가 끝나면 URL을 로드하고, 실패하면 에러 메시지를 보여줍니다.

29. 이 파일 전체를 다시 한 번 흐름으로 요약

- 앱이 시작되면 CanvasApp이 실행되고 CanvasHomePage가 첫 화면으로 열린다.
- 사용자는 배경 이미지를 올리거나 프레임 크기를 설정한다.
- 캔버스에서 line, pen, circle, rect 모드로 경로를 그린다.
- 앱은 그 경로를 일정 간격 점들로 재계산한다.
- 화면 좌표를 이미지 좌표로 바꾸고, 다시 mm 좌표로 변환한다.
- 점마다 Z 값을 붙인다.
- 필요하면 stroke를 선택, 이동, 스무딩, 점 편집한다.
- Send 버튼을 누르면 JSON payload를 만들고 TCP로 서버에 전송한다.

30. 초보자 관점에서 꼭 기억할 핵심 10개

- CanvasApp은 앱 껍데기, CanvasHomePage는 실제 작업 화면입니다.
- StatefulWidget이라서 화면 상태가 계속 변할 수 있습니다.
- _가 붙은 변수와 함수는 이 파일 내부용입니다.
- Offset은 좌표 한 점입니다.

- Stroke는 한 번 그린 선 묶음입니다.
- _rebuildPoints()가 "그림"을 "전송할 점 데이터"로 바꾸는 핵심 함수입니다.
- _canvasToImageClamped()와 _imageToCanvas()가 좌표계를 연결합니다.
- _ensureConnected()가 소켓 연결을 관리합니다.
- _buildPayload()가 최종 JSON을 만듭니다.
- LinePainter는 데이터를 실제 화면에 그리는 전용 클래스입니다.

생성 문서: main_android.dart 쉬운 해설

목적: 코드 조심자용 구조 이해, 변수와 함수가 왜 존재하는지 빠르게 파악